Log-rank conjecture for Boolean functions composed with OR and XOR

Yaroslav Alekseev, Yuval Filmus, Alexander V. Smal

doi:10.1007/s00037-025-00276-5; CC BY 4.0

Editorial note. Faithful editable transcription of actual CC BY4.0 complete journal version. Full author Lemma5.18 proof and Theorem5.17 final argument are retained, including the original OR substitution formula and coefficients. Standard cited degree/rank facts remain exact prerequisites. This is the author independent log-rank byproduct main result; support Lemma5.18 is not separately counted. No author formula is repaired or supplemented.

Allowed standard prerequisites (editorial).

- Decision-tree depth versus polynomial degree. DT(f) <= 2 deg(f)^4. Buhrman and de Wolf2002; exact invoked inequality retained
- Fourier sparsity equals XOR communication rank. Bernasconi and Codenotti1999; exact use retained in Lemma5.18
- Log rank is a deterministic communication lower bound. Lovasz and Saks1993; exact invoked inequality retained

Block composition (author source p.2)

For $f : \{0,1\}^n \to V$ and $g : \{0,1\}^m \to \{0,1\}$, a (block-)composition $f \diamond g : \{0,1\}^{n\times m} \to V$ is defined by

$$(f \diamond g)(z_1, z_2, \ldots, z_n) := f(g(z_1), g(z_2), \ldots, g(z_n)),$$

where each $z_i \in \{0,1\}^m$.

Prerequisites (author source pp.10–12)

All the logarithms are base 2. For $g : \{0,1\}^k \to \{0,1\}$, we define a function $g^n : \{0,1\}^{n\times k} \to \{0,1\}^n$ such that

$$g^n(x_1, x_2, \ldots, x_n) := (g(x_1), g(x_2), \ldots, g(x_n)).$$

For a function $f : \{0,1\}^n \to \{0,1\}$, an input $x \in \{0,1\}^n$, and a partial assignment $\sigma \in \{0,1,*\}^n$ that agrees with x on all non-$*$ coordinates, we use $f|_\sigma$ to denote a restriction of f to σ and $x|_\sigma$ to denote a projection of x to the $*$-coordinates of σ.

Decision tree complexity. A decision tree is a rooted binary tree with internal nodes labeled by input variables (representing queries), edges labeled with 0 and 1, and leaves labeled by some values. Decision tree evaluation is defined in a natural way: given an assignment for the input variables, we traverse the tree starting from the root and then sequentially choose every next edge according to the given assignments until we reach some leaf. The label of this leaf is the result of the evaluation. A decision tree computes some function f if for all possible assignments to the input variables, the decision tree evaluates to the value of the function. For a function f, $\mathrm{DT}(f)$ denotes the minimal depth of a decision tree computing f, and $\mathrm{DTSize}(f)$ denotes the minimal number of leaves in a decision tree computing f.

Certificate complexity. A certificate complexity is a non-deterministic analogue of the decision tree complexity. A certificate for an input $x \in \{0,1\}^n$ to a function f is a set $S \subseteq [n]$ of indices such that f restricted to all inputs that match x on S is constant, i.e., $f(y) = f(x)$ whenever $y|_S = x|_S$. The certificate complexity $\mathrm{C}(f,x)$ of f at input x is the size of the smallest certificate for x. Finally, the certificate complexity of function f is defined as $\mathrm{C}(f) := \max_{x\in\{0,1\}^n} \mathrm{C}(f,x)$.

Complexity definitions (author source pp.11–13)

Parity decision tree complexity. A parity decision tree extends the concept of a decision tree by allowing queries that are arbitrary linear combinations of the input variables. It can be described as a rooted binary tree with internal nodes labeled by linear combinations of the input variables, edges labeled with 0 and 1, and leaves labeled by some values. The evaluation is defined analogously. For a function f, $\mathrm{DT}_{\oplus}(f)$ denotes the minimal depth of a parity decision tree computing f, and $\mathrm{DTSize}_{\oplus}(f)$ denotes the minimal number of leaves in a parity decision tree computing f.

Degree and sparsity of the function. Every Boolean function defined on $\{0,1\}^n \to \{0,1\}$ can be considered as a function $f : \{-1,1\}^n \to \{-1,1\}$ by substituting 0 and 1 with 1 and -1, respectively. Every such function has a unique representation as a multilinear polynomial over $\mathbb{R}$ of the following form

$$f(x) = \sum_{S\subseteq[n]} c_S x_S, \qquad \text{where } x_S = \prod_{i\in S} x_i.$$

The degree of f is defined as the degree of the corresponding polynomial, and we denote it by $\deg(f)$. The sparsity of f is the number of nonzero coefficients in this representation, and we denote it by $\mathrm{spar}(f)$.

Deterministic communication complexity and rank. For a function $f : X \times Y \to Z$, consider the following game for two players, Alice and Bob, that want to compute f. Alice and Bob are given $x \in X$ and $y \in Y$, respectively. Their goal is to compute $f(x,y)$. In order to do it, the players exchange information about their parts of the input using a simple communication channel that allows sending bit messages. Before the game, the players come up with a

communication protocol that determines their behavior on all possible inputs. The cost of a protocol is the maximum total number of bits sent by the players over all possible inputs $x \in X, y \in Y$. The deterministic communication complexity of f is the minimal cost of a deterministic communication protocol that computes f. We denote it by $D(f)$. For a function $f : X \times Y \to \{0,1\}$, a communication matrix of f is a matrix $M_f \in \{0,1\}^{X \times Y}$ such that $(M_f)_{x,y} := f(x,y)$ for all $x \in X, y \in Y$. We define $\mathrm{rank}(f)$ to be the rank of the matrix M_f over the reals.

XOR functions (author source p.33)

In this section, we will need a composition of a function and a gadget (XOR in this case) in the communication complexity context. For a function $f : \{0,1\}^n \to \{0,1\}$ we define a function $f_\oplus : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}$, such that $f_\oplus(x,y) := f(x_1 \oplus y_1, \ldots, x_n \oplus y_n)$, and associate it with the following communication problem: Alice and Bob are given x and y, respectively, and their goal is to compute $f(x,y)$. For a subclass of such XOR functions, we will prove the following version of log-rank conjecture:

Theorem 5.17 (author source p.33). *For any function $f : \{0,1\}^n \to \{0,1\}$,*

$$D((f \diamond \mathrm{OR})_\oplus) \le \mathrm{poly}(\log \mathrm{rank}((f \diamond \mathrm{OR})_\oplus)).$$

Complete author proof (author source pp.34–35)

The proof depends on the following lemma.

Lemma 5.18. *For any function $f : \{0,1\}^n \to \{0,1\}$,*

$$\begin{aligned}\mathrm{DT}(f) &\overset{(1)}{\le} 2\deg(f)^4 \overset{(2)}{\le} O((\log \mathrm{spar}(f \diamond \mathrm{OR}))^4) \\ &\overset{(3)}{=} O((\log \mathrm{rank}((f \diamond \mathrm{OR})_\oplus))^4) \overset{(4)}{\le} O(D((f \diamond \mathrm{OR})_\oplus)^4) \\ &\overset{(5)}{\le} O(\mathrm{DT}_\oplus(f \diamond \mathrm{OR})^4).\end{aligned}$$

Proof. (1) See Buhrman & de Wolf (2002).

(2) We need to prove that

$$\deg(f) \le \log \mathrm{spar}(f \diamond \mathrm{OR}).$$

We can assume that $f : \{-1,1\}^n \to \{-1,1\}$ has degree n. If not, we can fix a monomial of the maximum degree and substitute 1 into all other variables. This substitution does not change the degree, while the sparsity can only decrease. So, the function f has the following multilinear representation:

$$f(x) = \alpha_{[n]} \prod_{i=1}^{n} x_i + \sum_{S \subsetneq [n]} \alpha_S x_S, \qquad \text{where } \alpha_{[n]} \neq 0.$$

In order to get the multilinear representation of the function $f \diamond \mathrm{OR}$ one can substitute each x_i with $\frac{y_{i1}y_{i2}-y_{i1}-y_{i2}-1}{2}$. After the substitution, for any combination of indices $j_1, j_2, \ldots, j_n \in \{1, 2\}$, the coefficient of the monomial $\prod_{k=1}^{n} y_{kj_k}$ is equal to $\alpha_{[n]} \cdot 2^{-n}$. Indeed, such monomials can only be produced by the substitution into the maximum degree monomial, and such substitution results in the coefficient $\alpha_{[n]} \cdot 2^{-n}$. Note that $\alpha_{[n]} \neq 0$ and hence there is a monomial for every 2^n possible combinations of indexes $j_1, j_2, \ldots, j_n$. That gives us the desired bound:

$$\log \mathrm{spar}(f \diamond \mathrm{OR}) \geq n = \deg(f).$$

(3) See Bernasconi & Codenotti (1999).

(4) See Lovász & Saks (1993).

(5) Each time we query a linear function $\ell(x \oplus y) = \ell(x) \oplus \ell(y)$, Alice and Bob can send to each other $\ell(x)$ and $\ell(y)$ to compute $\ell(x \oplus y)$.

□

Theorem 5.16 follows immediately. For Theorem 5.17, we need one more inequality in the chain.

Theorem 5.17. We need to show that

$$\mathrm{DT}_{\oplus}(f \diamond \mathrm{OR}) \leq 2\,\mathrm{DT}(f).$$

Indeed, the decision tree for f can be transformed into a decision tree for $f \diamond \mathrm{OR}$ by replacing every query with two queries of corresponding variables. Together with Lemma 5.18 that finishes the proof. □

References

Source reference used as standard background

Robert Beals, Harry Buhrman, Richard Cleve, Michele Mosca and Ronald de Wolf (2001). Quantum Lower Bounds by Polynomials. J. ACM 48(4), 778–797. doi:10.1145/502090.502097. Ankit Garg, Mika Göös, Pritish Kamath and Dmitry Sokolov (2020). Monotone circuit lower bounds from Resolution. Theory of Computing 16, Paper No.13. doi:10.4086/toc.2020.v016a013. Harry Buhrman and Ronald de Wolf (2002). Complexity measures and decision tree complexity: a survey. Theoretical Computer Science 288(1), 21–43. A. Bernasconi and B. Codenotti (1999). Spectral analysis of Boolean functions as a graph eigenvalue problem. IEEE Transactions on Computers 48(3), 345–351. László Lovász and Michael Saks (1993). Communication complexity and combinatorial lattice theory. Journal of Computer and System Sciences 47(2), 322–349.